SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE	DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

Program Name: B. Tech	Assignment Type: Lab	Academic Year:2025-2026

Course Coordinator Name	Dr. Rishabh Mittal
Instructor(s) Name	Mr. S Naresh Kumar Ms. B. Swathi Dr. Sasanko Shekhar Gantayat Mr. Md Sallauddin Dr. Mathivanan Mr. Y Srikanth Ms. N Shilpa Dr. Rishabh Mittal (Coordinator) Dr. R. Prashant Kumar Mr. Ankushavali MD Mr. B Viswanath Ms. Sujitha Reddy Ms. A. Anitha Ms. M.Madhuri Ms. Katherashala Swetha Ms. Velpula sumalatha Mr. Bingi Raju

CourseCode	23CS002PC304	**Course Title**	AI Assisted Coding
Year/Sem	III/II	**Regulation**	R23
Date and Day of Assignment	**Week 6 – Friday**	**Time(s)**	23CSBTB01 To 23CSBTB52
Duration	2 Hours	**Applicable to Batches**	All batches

Assignment Number: 12.5(Present assignment number)/**24**(Total number of assignments)

Q.No.	Question	*Expected Time to complete*

Lab 12: Algorithms with AI Assistance – Sorting, Searching, and Optimizing Algorithms

Lab Objectives:

- Apply AI-assisted programming to implement and optimize sorting and searching algorithms.
- Compare different algorithms in terms of efficiency and use cases.
- Understand how AI tools can suggest optimized code and complexity improvements.

1

Task Description #1 (Sorting – Merge Sort Implementation)

- Task: Use AI to generate a Python program that implements the Merge Sort algorithm.
- Instructions:
  - Prompt AI to create a function merge_sort(arr) that sorts a list in ascending order.
  - Ask AI to include time complexity and space complexity in the function docstring.
  - Verify the generated code with test cases.
- Expected Output:
  - A functional Python script implementing Merge Sort with proper documentation.

Week6 -Friday

PROMPT

```
#Generate a Python function merge_sort(arr) that sorts a list in ascending order.Include time complexity and space complexity in the docstring.Add test cases.
```

CODE

```
def merge_sort(arr):

    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    return merge(left, right)


def merge(left, right):
```

```python
    sorted_list = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            sorted_list.append(left[i])
            i += 1
        else:
            sorted_list.append(right[j])
            j += 1

    sorted_list.extend(left[i:])
    sorted_list.extend(right[j:])
    return sorted_list


# Test Case
arr = [38, 27, 43, 3, 9, 82, 10]
print("Sorted Array:", merge_sort(arr))
```

CODE OP

Sorted Array: [3, 9, 10, 27, 38, 43, 82]

EXPLANATION

Merge Sort uses the Divide and Conquer technique:

1. Divide the list into halves.
2. Recursively sort both halves.
3. Merge the sorted halves.

It always runs in **O(n log n)** time and requires **O(n)** extra space.

Task Description #2 (Searching – Binary Search with AI Optimization)

- Task: Use AI to create a binary search function that finds a target element in a sorted list.
- Instructions:
  - Prompt AI to create a function binary_search(arr, target) returning the index of the target or -1 if not found.
  - Include docstrings explaining best, average, and worst-

case complexities.
    - Test with various inputs.
- Expected Output:
    - Python code implementing binary search with AI-generated comments and docstrings.

PROMPT `#Create a Python function binary_search(arr, target) that returnsthe index of target in a sorted list or -1 if not found.Include best, average, and worst-case complexities in docstring.`

CODE

```python
def binary_search(arr, target):

    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2

        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1

    return -1


# Test Case
arr = [3, 9, 10, 27, 38, 43, 82]
print("Index of 27:", binary_search(arr, 27))
print("Index of 100:", binary_search(arr, 100))
```

CODE OP

Index of 27: 3

Index of 100: -1

EXPLANATION

1. Find middle element.
2. If target equals middle → return index.

3. If target smaller → search left half.
4. If target larger → search right half.

Time complexity is **O(log n)** which is much faster than linear search O(n).

Task Description #3: Smart Healthcare Appointment Scheduling System

A healthcare platform maintains appointment records containing appointment ID, patient name, doctor name, appointment time, and consultation fee. The system needs to:

1. Search appointments using appointment ID.
2. Sort appointments based on time or consultation fee.

Student Task

- Use AI to recommend suitable searching and sorting algorithms.
- Justify the selected algorithms.
- Implement the algorithms in Python.

PROMPT

```
#Recommend suitable searching and sorting
algorithms.Justify the selection.Implement searching by
appointment ID and sorting by time or fee.
```

CODE

```
appointments = [
    {"id": 101, "patient": "A", "time": 10, "fee": 500},
    {"id": 102, "patient": "B", "time": 8, "fee": 300},
    {"id": 103, "patient": "C", "time": 12, "fee": 700}
]

# Sort by time
appointments.sort(key=lambda x: x["time"])
```

```
print("Sorted by Time:", appointments)

# Binary Search by ID
def search_appointment(data, target):
    data.sort(key=lambda x: x["id"])
    low, high = 0, len(data)-1

    while low <= high:
        mid = (low+high)//2
        if data[mid]["id"] == target:
            return data[mid]
        elif data[mid]["id"] < target:
            low = mid+1
        else:
            high = mid-1
    return "Not Found"

print("Search Result:", search_appointment(appointments,
102))
```

CODE OP

Sorted by Time: [{'id': 102, 'patient': 'B', 'time': 8, 'fee': 300}, {'id': 101, 'patient': 'A', 'time': 10, 'fee': 500}, {'id': 103, 'patient': 'C', 'time': 12, 'fee': 700}]

Search Result: {'id': 102, 'patient': 'B', 'time': 8, 'fee': 300}

EXPLANATION

Binary search reduces search time.

Sorting allows better scheduling and management.

Justification

Binary Search is efficient for searching appointment ID.
Merge Sort is stable and good for sorting by time or fee.

Task Description #4: Railway Ticket Reservation System

Scenario

A railway reservation system stores booking details such as ticket ID,

passenger name, train number, seat number, and travel date. The system must:

1. Search tickets using ticket ID.
2. Sort bookings based on travel date or seat number.

Student Task

- Identify efficient algorithms using AI assistance.
- Justify the algorithm choices.
- Implement searching

PROMPT

```
#"Suggest efficient searching and sorting algorithms for a
Railway Reservation System that searches tickets by ticket
ID and sorts bookings by travel date or seat number.
Implement in Python with justification."
```

CODE

```
tickets = [
    {"ticket_id": 101, "name": "Mukesh", "train_no": 1234,
"seat": 45, "date": "2026-03-01"},
    {"ticket_id": 102, "name": "Rahul", "train_no": 1235,
"seat": 12, "date": "2026-02-28"},
    {"ticket_id": 103, "name": "Anita", "train_no": 1236,
"seat": 30, "date": "2026-03-05"}
]

# Binary Search (after sorting by ticket_id)
def binary_search(arr, target):
    low, high = 0, len(arr)-1
    while low <= high:
        mid = (low + high)//2
        if arr[mid]["ticket_id"] == target:
            return arr[mid]
        elif arr[mid]["ticket_id"] < target:
            low = mid + 1
        else:
            high = mid - 1
    return None
```

```python
# Sort by date
sorted_by_date = sorted(tickets, key=lambda x: x["date"])

# Sort by seat
sorted_by_seat = sorted(tickets, key=lambda x: x["seat"])

tickets.sort(key=lambda x: x["ticket_id"])
result = binary_search(tickets, 102)

print("Search Result:", result)
print("Sorted by Date:", sorted_by_date)
print("Sorted by Seat:", sorted_by_seat)
```

CODE OP

PS C:\Ai_coding>

Search Result: {'ticket_id': 102, 'name': 'Rahul', 'train_no': 1235, 'seat': 12, 'date': '2026-02-28'}

Sorted by Date: [{'ticket_id': 102, 'name': 'Rahul', 'train_no': 1235, 'seat': 12, 'date': '2026-02-28'}, {'ticket_id': 101, 'name': 'Mukesh', 'train_no': 1234, 'seat': 45, 'date': '2026-03-01'}, {'ticket_id': 103, 'name': 'Anita', 'train_no': 1236, 'seat': 30, 'date': '2026-03-05'}]

Sorted by Seat: [{'ticket_id': 102, 'name': 'Rahul', 'train_no': 1235, 'seat': 12, 'date': '2026-02-28'}, {'ticket_id': 103, 'name': 'Anita', 'train_no': 1236, 'seat': 30, 'date': '2026-03-05'}, {'ticket_id': 101, 'name': 'Mukesh', 'train_no': 1234, 'seat': 45, 'date': '2026-03-01'}]

EXPLANATION

- Binary Search reduces search time significantly.
- Sorting by date/seat improves booking organization.
- Efficient for large railway databases.

Task Description #5: Smart Hostel Room Allocation System

A hostel management system stores student room allocation details including student ID, room number, floor, and allocation date. The system needs to:

1. Search allocation details using student ID.
2. Sort records based on room number or allocation date.

Student Task

- Use AI to suggest optimized algorithms.
- Justify the selections.
- Implement the solution in Python.

PROMPT

```
#"Suggest optimized algorithms for searching student ID
and sorting by room number or allocation date. Implement
in Python."
```

CODE

```
hostel = [
    {"student_id": 201, "room": 101, "floor": 1, "date":
"2026-01-15"},
    {"student_id": 202, "room": 205, "floor": 2, "date":
"2026-01-10"},
    {"student_id": 203, "room": 103, "floor": 1, "date":
"2026-01-20"}
]

def binary_search(arr, target):
    low, high = 0, len(arr)-1
    while low <= high:
```

```
        mid = (low + high)//2
        if arr[mid]["student_id"] == target:
            return arr[mid]
        elif arr[mid]["student_id"] < target:
            low = mid + 1
        else:
            high = mid - 1
    return None

hostel.sort(key=lambda x: x["student_id"])
print("Search:", binary_search(hostel, 202))

print("Sort by Room:", sorted(hostel, key=lambda x:
x["room"]))
print("Sort by Date:", sorted(hostel, key=lambda x:
x["date"]))
```

CODE OP

Search: {'student_id': 202, 'room': 205, 'floor': 2, 'date': '2026-01-10'}

Sort by Room: [{'student_id': 201, 'room': 101, 'floor': 1, 'date': '2026-01-15'}, {'student_id': 203, 'room': 103, 'floor': 1, 'date': '2026-01-20'}, {'student_id': 202, 'room': 205, 'floor': 2, 'date': '2026-01-10'}]

Sort by Date: [{'student_id': 202, 'room': 205, 'floor': 2, 'date': '2026-01-10'}, {'student_id': 201, 'room': 101, 'floor': 1, 'date': '2026-01-15'}, {'student_id': 203, 'room': 103, 'floor': 1, 'date': '2026-01-20'}]

EXPLANATION

- Fast student lookup.
- Sorting helps hostel management allocate efficiently.
- Stable and scalable.

Task Description #6: Online Movie Streaming Platform

A streaming service maintains movie records with movie ID, title, genre, rating, and release year. The platform needs to:

1. Search movies by movie ID.
2. Sort movies based on rating or release year.

Student Task

- Recommend searching and sorting algorithms using AI.
- Justify the chosen algorithms.
- Implement Python functioN
- **PROMPT**
- #"Recommend searching and sorting algorithms for movie ID search and sorting by rating or release year."
- **CODE**
- movies = [

```
    {"movie_id": 1, "title": "Inception", "genre": "Sci-Fi", "rating": 8.8, "year": 2010},
    {"movie_id": 2, "title": "Avatar", "genre": "Fantasy", "rating": 7.8, "year": 2009},
    {"movie_id": 3, "title": "Interstellar", "genre": "Sci-Fi", "rating": 8.6, "year": 2014}
]

def binary_search(arr, target):
    low, high = 0, len(arr)-1
    while low <= high:
        mid = (low + high)//2
        if arr[mid]["movie_id"] == target:
            return arr[mid]
        elif arr[mid]["movie_id"] < target:
            low = mid + 1
        else:
            high = mid - 1
    return None

movies.sort(key=lambda x: x["movie_id"])

```

```
• print("Search:", binary_search(movies, 3))
• print("Sort by Rating:", sorted(movies, key=lambda
  x: x["rating"], reverse=True))
• print("Sort by Year:", sorted(movies, key=lambda x:
  x["year"]))
```

- **CODE OP**
- Search: {'movie_id': 3, 'title': 'Interstellar', 'genre': 'Sci-Fi', 'rating': 8.6, 'year': 2014}
- Sort by Rating: [{'movie_id': 1, 'title': 'Inception', 'genre': 'Sci-Fi', 'rating': 8.8, 'year': 2010}, {'movie_id': 3, 'title': 'Interstellar', 'genre': 'Sci-Fi', '
- rating': 8.6, 'year': 2014}, {'movie_id': 2, 'title': 'Avatar', 'genre': 'Fantasy', 'rating': 7.8, 'year': 2009}]
- Sort by Year: [{'movie_id': 2, 'title': 'Avatar', 'genre': 'Fantasy', 'rating': 7.8, 'year': 2009}, {'movie_id': 1, 'title': 'Inception', 'genre': 'Sci-Fi', 'rating': 8.8, 'year': 2010}, {'movie_id': 3, 'title': 'Interstellar', 'genre': 'Sci-Fi', 'rating': 8.6, 'year': 2014}]
-

Task Description #7: Smart Agriculture Crop Monitoring System

An agriculture monitoring system stores crop data with crop ID, crop name, soil moisture level, temperature, and yield estimate. Farmers need to:

1. Search crop details using crop ID.
2. Sort crops based on moisture level or yield estimate.

Student Task

- Use AI-assisted reasoning to select algorithms.
- Justify algorithm suitability.
- Implement searching and sorting in Python.

PROMPT

```
#Select algorithms for searching crop ID and sorting by
moisture level or yield estimate."
```

CODE

```python
crops = [
    {"crop_id": 11, "name": "Rice", "moisture": 30,
"yield": 50},
    {"crop_id": 12, "name": "Wheat", "moisture": 25,
"yield": 45},
    {"crop_id": 13, "name": "Corn", "moisture": 35,
"yield": 60}
]

def binary_search(arr, target):
    low, high = 0, len(arr)-1
    while low <= high:
        mid = (low + high)//2
        if arr[mid]["crop_id"] == target:
            return arr[mid]
        elif arr[mid]["crop_id"] < target:
            low = mid + 1
        else:
            high = mid - 1
    return None

crops.sort(key=lambda x: x["crop_id"])

print("Search:", binary_search(crops, 12))
print("Sort by Moisture:", sorted(crops, key=lambda x:
x["moisture"]))
print("Sort by Yield:", sorted(crops, key=lambda x:
x["yield"], reverse=True))
```

CODE OP

Search: {'crop_id': 12, 'name': 'Wheat', 'moisture': 25, 'yield': 45}

Sort by Moisture: [{'crop_id': 12, 'name': 'Wheat', 'moisture': 25, 'yield': 45}, {'crop_id': 11, 'name': 'Rice', 'moisture': 30, 'yield': 50}, {'crop_id': 13, 'name': 'Corn', 'moisture': 35, 'yield': 60}]

Sort by Yield: [{'crop_id': 13, 'name': 'Corn', 'moisture': 35, 'yield': 60}, {'crop_id': 11, 'name': 'Rice', 'moisture': 30, 'yield': 50}, {'crop_id': 12, 'name': 'Wheat', 'moisture': 25, 'yield': 45}]

EXPLANATION

- Efficient for farm monitoring dashboards.
- Sorting helps identify high-yield crops.

- Useful for decision-making systems.

Task Description #8: Airport Flight Management System

An airport system stores flight information including flight ID, airline name, departure time, arrival time, and status. The system must:

1. Search flight details using flight ID.
2. Sort flights based on departure time or arrival time.

Student Task

- Use AI to recommend algorithms.
- Justify the algorithm selection.
- Implement searching and sorting logic in Python.
- **PROMPT**
- #"Recommend algorithms for searching flight ID and sorting by departure or arrival time."
- **CODE**
- flights = [

```
    {"flight_id": 501, "airline": "Indigo", "departure": "10:00", "arrival": "12:00", "status": "On Time"},
    {"flight_id": 502, "airline": "Air India", "departure": "08:00", "arrival": "10:30", "status": "Delayed"},
    {"flight_id": 503, "airline": "SpiceJet", "departure": "14:00", "arrival": "16:00", "status": "On Time"}
]

def binary_search(arr, target):
    low, high = 0, len(arr)-1
    while low <= high:
        mid = (low + high)//2
        if arr[mid]["flight_id"] == target:
            return arr[mid]
        elif arr[mid]["flight_id"] < target:
            low = mid + 1
        else:
            high = mid - 1
```

```
    return None

flights.sort(key=lambda x: x["flight_id"])

print("Search:", binary_search(flights, 502))
print("Sort by Departure:", sorted(flights, key=lambda x: x["departure"]))
print("Sort by Arrival:", sorted(flights, key=lambda x: x["arrival"]))
```

- **CODE OP**
- Search: {'flight_id': 502, 'airline': 'Air India', 'departure': '08:00', 'arrival': '10:30', 'status': 'Delayed'}
- Sort by Departure: [{'flight_id': 502, 'airline': 'Air India', 'departure': '08:00', 'arrival': '10:30', 'status': 'Delayed'}, {'flight_id': 501, 'airline': 'Indigo', 'departure': '10:00', 'arrival': '12:00', 'status': 'On Time'}, {'flight_id': 503, 'airline': 'SpiceJet', 'departure': '14:00', 'arrival': '16:00', 'status': 'On Time'}]
- Sort by Arrival: [{'flight_id': 502, 'airline': 'Air India', 'departure': '08:00', 'arrival': '10:30', 'status': 'Delayed'}, {'flight_id': 501, 'airline': 'Indigo', 'departure': '10:00', 'arrival': '12:00', 'status': 'On Time'}, {'flight_id': 503, 'airline': 'SpiceJet', 'departure': '14:00', 'arrival': '16:00', 'status': 'On Time'}]
- **EXPLANATION**
- Binary search ensures quick flight lookup.
- Sorting by departure helps scheduling.
- Sorting by arrival helps tracking delays.
-